Session 16 — Study Guide: From Who to What

Where the session ends

Real, workspace-scoped, role-based authorization gates every write route on Projects and Workspaces (five routes, one shared, reusable mechanism, three different ways of finding the relevant workspace). Real, ownership-based authorization gates every write route on Issues, built a genuinely different way. Authentication (Session 15) and authorization (this session) are both real, both proven, and clearly distinguished.

The real starting point: a foundation that had to be rebuilt

Session 15's own homework — a real `WorkspaceMember` table recording each user's role in each workspace — was assigned the day before this session and, honestly, is very unlikely to exist on your own machine yet. This session opens by building it together, live, as real content — not a side detour. Along the way, a real gap in the app's own data model was also closed: `Project` never actually recorded which `Workspace` it belonged to, even though the written design always said it should.

The problem: a verified identity still isn't a decision

A `JwtAuthGuard`-protected route already knows exactly who is asking. It knows nothing about whether that specific person should be allowed to do this specific thing. **Authentication** answers "who are you?" **Authorization** answers "what are you allowed to do?" — a completely separate question, answered by completely separate code.

401 vs. 403 — a precise distinction, not two words for "no"

- **401 Unauthorized** — your IDENTITY could not be established (no token, bad token, expired token).
- **403 Forbidden** — your identity IS established, and the answer is still no.

Before this session, this app could only ever say 401 or succeed. After today, a real, signed-in, perfectly valid user can get a real 403 for the first time — because now something actually checks what they're allowed to do, not just who they are.

Why role can never live in the JWT

A token's claims are frozen the instant it's signed. If your role were written into your token at login, a role CHANGE after that (a promotion, a demotion, being removed) would be invisible until you logged in again — a real gap, not a nitpick. Every authorization check in this app instead re-reads the CURRENT `WorkspaceMember` row from the database, every single time.

A concrete guard, built once, understood deeply

Live code built `AdminOrOwnerGuard`: it reads the signed-in user, reads a `workspaceId` from the request, looks up that user's real membership, and allows the request only if they're a real Owner or Admin there. One guard, one route (`POST /projects`), one hardcoded role set. Deliberately simple, so the MECHANISM is fully understood before anyone is asked to generalize it.

Teamwork: making it reusable (you discover this — it is not demonstrated live)

Four more real routes each needed similar protection, but with different role requirements and different ways of finding the relevant workspace. Rather than writing four near-duplicate guards, teamwork researches and builds a genuinely reusable mechanism: **custom route metadata** (attached with a decorator built on `SetMetadata`) read by **one shared guard** (via `Reflector`). The same guard class ends up protecting `POST /projects`, `PATCH` / `DELETE /projects/:id`, and `PATCH` / `DELETE /workspaces/:id` — each configured differently, through metadata, not through separate code.

Homework: a rule that genuinely isn't a role check

`AUTH-AND-PERMISSIONS.md` has one deliberately different rule: an Issue may only be edited or deleted by its own **assignee**, its own **reporter**, or a workspace **Admin/Owner**. Neither "assignee" nor "reporter" is a role — they're facts about ONE SPECIFIC issue. A guard COULD be built for this (it could load the Issue itself and resolve the same chain teamwork's own guard already resolves for Projects/Workspaces) — but this app chooses the service layer instead, since the service already has to load the specific issue being mutated, and checking the rule there avoids loading it twice. This is this session's own homework, and it is a genuinely different authorization SHAPE by deliberate choice, not a smaller version of the same one, and not something a guard is incapable of.

Not every route needs protecting — and that's still true today

Every `GET` route in this app remains open, exactly as before. `GET /projects/stats` still requires only the old internal-tool key, exactly as Session 15 left it. Authorization this session is about WRITE routes specifically, matching the real rules in `AUTH-AND-PERMISSIONS.md` — not a blanket "lock everything down."

Common beginner mistakes

See `student/notes/STUDENT_NOTES-EN.md`.

A preview of what's still missing

There is still no way, anywhere in this app, to actually invite someone to a workspace or change their role through a real endpoint — every membership fact this session uses was seeded directly into the database for proof purposes. That's real, deliberate, future work, not something this session forgot.

Not yet

- An invite/add-member endpoint, a role-change endpoint, or any route that reads/lists `WorkspaceMember` rows directly.
- The agile-planning layer (iteration containers and estimation points), discussion threads on issues, tagging other users by name inside text, the activity and notification feed.
- PostgreSQL, an ORM, Next.js.

Glossary

- **Authentication** — verifying who is making a request.
- **Authorization** — deciding what an identified requester may do.
- **Role-based authorization** — a decision based on a role value alone (Owner/Admin/Member).
- **Ownership-based authorization** — a decision based on a specific resource's own fields (assignee, reporter), not a role alone.
- **Custom route metadata** — data attached to a route/class via `SetMetadata()`, read later by a guard via `Reflector`.
- **`Reflector`** — a Nest utility a guard uses to read metadata attached to the current route.

Review questions

1. Why can a perfectly valid, signed-in user still receive a 403 today, when before today they never could?
2. Why does the authorization guard re-query the database instead of trusting the JWT's own claims?
3. What does `@RequireWorkspaceRole()` actually DO by itself, with no guard reading it?
4. Why do `PATCH /projects/:id` and `PATCH /workspaces/:id` resolve their own workspace id differently, despite using the identical decorator and guard?
5. Could the Issue ownership rule technically be written as a guard? Why does this app choose not to?

Answers

1. Because something now checks WHAT they're allowed to do, not just WHO they are — identity alone used to be the only question this app could ask.
2. Because a token's claims are frozen at signing time; a role change after login would otherwise be invisible until the user logged in again.
3. Nothing by itself — it only attaches inert metadata to the route/class; a guard reading it back via `Reflector` is what makes it actually matter.
4. Because each route's own URL shape is different: a Workspace route's own `:id` already IS the workspace id; a Project route's own `:id` is the PROJECT's id, so the guard must load that project first to find its own workspace.
5. Yes — a guard could load the Issue itself, resolve its Project, resolve its WorkspaceMember, and authorize, exactly like `WorkspaceRoleGuard` already does. This app chooses the service layer instead because the service already has to load that same Issue to perform the mutation — checking the rule there avoids loading it twice and keeps the rule beside the mutation it governs.

What you should be able to explain now

- The precise difference between 401 and 403, with a real example of each from this app.
- Why role truth lives in the database, never the token.
- What a guard, a custom decorator, and `Reflector` each actually do, and how the three work together.
- Why a guard-based mechanism fit Projects/Workspaces well, and why this app chose a different shape for the Issue ownership rule instead — as a design tradeoff, not because a guard couldn't do it.